1. What is Express.js

Definition

Express.js is a **minimal and flexible web application framework for Node.js** that simplifies the development of web servers and APIs.

It provides features such as:

- Routing
- Middleware support
- Request and response handling
- Template rendering
- REST API creation

Express removes the complexity of using Node.js core modules directly.

Why Express.js is Needed

Using only Node.js server requires writing more code.

Example using Node.js only

```
const http = require('http');

http.createServer((req,res)=>{
    res.write("Hello Students");
    res.end();
}).listen(3000);
```

This works but becomes complicated for large applications.

Express simplifies it.

Example using Express

```
const express = require('express');
const app = express();

app.get('/',(req,res)=>{
    res.send("Hello Students");
});

app.listen(3000);
```

Much simpler.

Real Life Example

Imagine a **hotel restaurant**.

Components:

Real Life	Web Application
Customer	Browser
Order	Request
Waiter	Express Server
Kitchen	Application Logic
Food	Response

The waiter (Express) manages communication between the customer and kitchen.

Key Points

- Express is built on Node.js
- It is used to build **web applications and APIs**
- It is lightweight and fast
- It supports middleware architecture

---

2. Setting Up Express.js Environment

Before using Express, we must prepare the development environment.

Step 1 Install Node.js

Install **Node.js** from the official website.

Check installation:

```
node -v
```

This confirms Node is installed.

Step 2 Create Project Folder

Create a folder for your project.

Example:

```
express-demo
```

Open it in **Visual Studio Code**.

Step 3 Initialize Node Project

Run:

```
npm init -y
```

This creates a file:

```
package.json
```

Purpose of package.json

It stores:

- project name
- version
- dependencies
- scripts

Example structure:

```
{
"name": "express-demo",
"version": "1.0.0",
"dependencies": {}
}
```

Step 4 Install Express

Run:

```
npm install express
```

This downloads Express and saves it inside:

```
node_modules
```

Step 5 Create Server File

Create:

```
server.js
```

Project structure:

```
express-demo
│
├── node_modules
├── package.json
└── server.js
```

---

3. Creating a Basic Server with Express

A **server listens for requests from clients and sends responses back**.

Basic Express Server

```
const express = require('express');

const app = express();

const PORT = 3000;

app.get('/', (req,res)=>{
    res.send("Welcome to Express Server");
});

app.listen(PORT, ()=>{
    console.log("Server running on port 3000");
});
```

Code Explanation

```
const express = require('express');
const app = express();

// server port
const PORT = 3000;

// route
app.get('/', (req, res) => {
  res.send('Hello Students!');
});

// start server
  app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Explanation (Line by Line)

- `require('express')` → import Express
- `app = express()` → create app (server)
- `app.get()` → define route
- `req` → request from user
- `res` → response to user
- `app.listen()` → start server

Testing the Server

Open browser:

```
http://localhost:3000
```

Output:

```
Welcome to Express Server
```

---

4. Routing in Express.js

Definition

Routing refers to **determining how an application responds to a client request for a specific URL and HTTP method**.

Components of Routing

1 URL path
2 HTTP method
3 Handler function

Example Routes

```
/        -> Home Page
/about   -> About Page
/contact -> Contact Page
```

Example Code

```
const express = require('express');
const app = express();

app.get('/',(req,res)=>{
    res.send("Home Page");
});

app.get('/about',(req,res)=>{
    res.send("About Page");
});

app.get('/contact',(req,res)=>{
    res.send("Contact Page");
});

app.listen(3000);
```

Real Life Example

Think of **departments in a college**.

Department	URL
Computer	/computer
Mechanical	/mechanical

Department	URL
Civil	/civil

Students go to different rooms based on department.

Routing works the same way.

Types of Routing Methods

Method	Purpose
GET	Retrieve data
POST	Send data
PUT	Update data
DELETE	Delete data

Example:

```
app.post('/student',(req,res)=>{
res.send("Student Added");
});
```

---

5. Middleware in Express.js

Definition

Middleware functions are functions that **execute during the lifecycle of a request before sending the final response**.

They have access to:

```
req
res
next
```

Middleware Flow

```
Client Request
      ↓
Middleware
```

```
      ↓
Route Handler
      ↓
Response
```

Example Middleware

```
const express = require('express');
const app = express();

app.use((req,res,next)=>{
    console.log("Middleware executed");
    next();
});

app.get('/',(req,res)=>{
    res.send("Home Page");
});

app.listen(3000);
```

Real Life Example

Airport security check:

```
Passenger → Security → Boarding
```

Security acts like middleware.

Types of Middleware

1 Application level middleware
2 Router level middleware
3 Built-in middleware
4 Third-party middleware

Example built-in middleware:

```
app.use(express.json());
```

Used to parse JSON data.

---

6. Handling Requests and Responses

Express applications handle **HTTP requests and responses**.

Request Object (req)

The request object contains information about the client request.

Examples:

- URL
- headers
- query parameters
- request body

Example:

```
app.get('/user',(req,res)=>{
console.log(req.query.name);
});
```

URL:

```
/user?name=shivani
```

Output:

```
shivani
```

Response Object (res)

The response object sends data back to the client.

Common response methods:

Method	Description
res.send()	send text
res.json()	send JSON
res.status()	send HTTP status
res.sendFile()	send files

Example

```
app.get('/student',(req,res)=>{
res.json({
name:"Shivani",
course:"Computer Engineering"
});
});
```

Output

```
{
"name":"Shivani",
"course":"Computer Engineering"
}
```

Query Parameters: -

```
app.get('/search', (req, res) => {
  const name = req.query.name;
  res.send(`Search for ${name}`);
});
```

URL :- /search?name=Job

---

7. Templating Engines

Definition

Template engines allow developers to **generate dynamic HTML pages using data from the server**.

Instead of static HTML, pages are created dynamically.

Popular Template Engines

- **EJS**
- **Pug**
- **Handlebars**

Why Template Engines Are Used

They help in:

- dynamic content
- code reusability
- separating logic from HTML

Example using EJS

Install EJS

```
npm install ejs
```

---

server.js

```
const express = require('express');
const app = express();

app.set('view engine','ejs');

app.get('/',(req,res)=>{
res.render('index',{name:"Shivani"});
});

app.listen(3000);
```

---

views/index.ejs

```
<h1>Hello <%= name %></h1>
```

Output

```
Hello Shivani
```

Additional Concepts

To give **industry-level understanding**, you can also briefly introduce:

Static Files

```
app.use(express.static('public'));
```

Used for CSS, images, JavaScript.

Express Router

Used to organize large applications.

Example:

```
routes
   user.js
   product.js
```

Complete Express Request Flow

Explain this in class:

```
Client (Browser)
      ↓
Request
      ↓
Express Server
      ↓
Middleware
      ↓
Routing
      ↓
Controller Logic
      ↓
Response
      ↓
Browser Output
```

Short Summary for Students

Express.js helps developers:

- build web servers
- manage routes
- process requests

- send responses
- create dynamic web pages

It is one of the **most popular frameworks in Node.js development**.